ATL DEV CON· 10/17/2026

$ cat .env # your agent can too

Stop Handing Your Secrets to an Agent

Secrets management for developers in the agent era

Jitendrudu · Sr Software Engineer · Timmons Group

THE SETUP

Your coding agent runs as you

It can read

~/.aws/credentials

~/.ssh

~/.npmrc

every .env in the repo

It can run

npm install, with your tokens in its environment

It remembers

Command output lands in its context, the transcript, and sometimes a commit

Attackers noticed before most of us did.

Thirteen months, one pattern

Aug 2025

s1ngularity (Nx)

Postinstall ran your installed AI CLIs with bypass flags to hunt for secrets

Sep 2025

Shai-Hulud

First self-spreading npm worm: steal the token, republish every package it reaches

Nov 2025

Shai-Hulud 2.0

Preinstall Bun loader, a bigger harvest, and a home wipe on failure

Dec 2025

npm revokes classic tokens

npm login becomes a 2-hour session; write tokens capped at 90 days

Apr 2026

Mini Shai-Hulud

Steals agent configs, then plants hooks so opening the repo in the agent re-runs it

May 2026

TanStack

84 malicious versions via OIDC with no token stolen, and valid SLSA provenance

Mini Shai-Hulud targeted the agent

1 · Install

Runs on install

A preinstall hook downloads Bun and runs an 11.6 MB obfuscated stealer. Bun slips past monitors that watch for node.

2 · Harvest

Takes everything

npm, AWS, GCP, Azure and GitHub tokens. 134 file paths, including **/.env, ~/.claude.json and ~/.claude/mcp.json.

3 · Exfiltrate

Hides in plain sight

Encrypts the loot and commits it to a new public repo on your own GitHub account, via api.github.com.

4 · Persist

Moves into your agent

Commits .claude/settings.json with a SessionStart hook and .vscode/tasks.json with runOn: folderOpen, signed as claude.

THE DATA

More code, more leaks, longer lives

28.65M

new hardcoded secrets on public GitHub in 2025, up 34%

3.2% vs 1.5%

leak rate of Claude Code-assisted commits versus the baseline

24,008

unique secrets exposed in MCP configuration files

64%

of secrets valid in 2022 were still valid in January 2026

Habits built for human-only terminals

Habit	What we assumed	What is true with agents
.env in the repo root	Only my app reads it	Every agent read, subprocess and install script can
Cloud keys in CI secrets	Only our workflow sees them	Any code in the job can read runner memory
A PAT for the release bot	A person is accountable	One leak reaches every repo that person can
"I'll review before I commit"	I write every line	The agent writes, commits and pushes in minutes
Repo config is inert	Config is data	Hooks and tasks run code when you open the folder

THE PLAN

Five places the old habits break

01	**Local development**	Why .env is a liability, and what replaces it
02	**CI/CD**	No cloud keys in GitHub Actions
03	**Tokens**	Short-lived and narrowly scoped
04	**AI agents**	Give it far less to leak
05	**Safety nets**	Catch it, and the plan for when you don't

That .env file has four readers you never invited

The agent's file tool

It reads the project to understand it. To a model, .env is context.

Every subprocess

Anything the agent runs inherits your files and environment.

Every install script

Mini Shai-Hulud harvests **/.env by glob. No layout knowledge needed.

The transcript

Whatever the agent read or printed now lives in a log.

Gitignore is not access control. It blocks one path: the commit.

01 · LOCAL DEVELOPMENT

Commit references. Resolve them at run time.

Before: .env

```
DATABASE_URL=postgres://app:9f3Kq…
STRIPE_SECRET_KEY=sk_live_51H…
OPENAI_API_KEY=sk-proj-Qm2x…
```

Readable by anything running as you

After: .env.tpl, safe to commit

```
DATABASE_URL=op://dev/app/db_url
STRIPE_SECRET_KEY=op://dev/app/stripe
OPENAI_API_KEY=op://dev/app/openai
$ op run --env-file=.env.tpl -- npm start
```

Values exist only inside the process you start

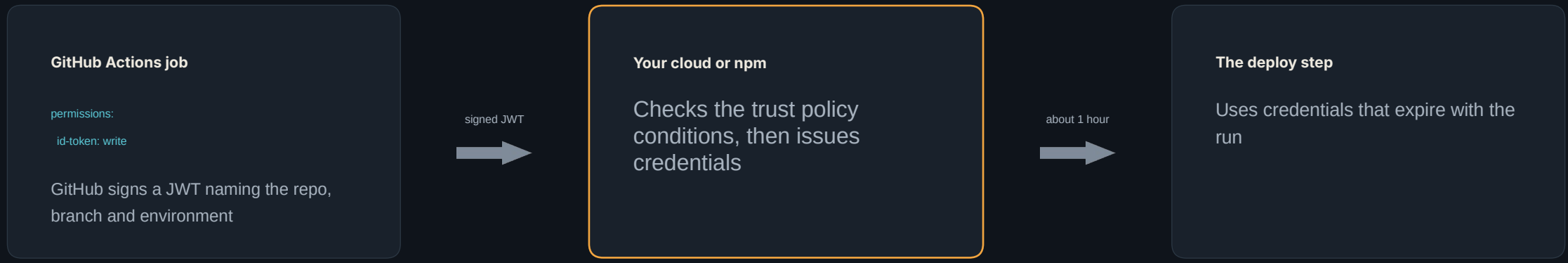
Same idea: doppler run · infisical run · aws-vault exec. And keep the resolving step outside the agent.

Demo 1: What an install script sees

```
$ npm run demo:install-unsafe
  # a harmless postinstall prints what it can read as me
$ npm run demo:install-safe
  # same install with --ignore-scripts: nothing runs
$ cat .env.tpl
  # references: safe to commit, safe for the agent to read
```

Watch for: my env var, every .env value, which credential files exist, which AI agents are installed.

No cloud keys in GitHub Actions



Nothing stored. Nothing to rotate. The trust policy is the whole control.

The trust policy is the control

```
"Condition": {
  "StringEquals": {
    "token.actions.githubusercontent.com:aud":
      "sts.amazonaws.com",
    "token.actions.githubusercontent.com:sub":
      "repo:acme/web:environment:production"
  }
}
```

Too wide

repo:acme/* or StringLike on sub lets any repo or branch in.

Right

The repo plus a protected environment with required reviewers.

npm too

Set the workflow file and an environment. SAP's @cap-js trusted the whole repo, so a side-branch workflow minted a publish token.

OIDC didn't save TanStack

1 A pull_request_target workflow ran fork code with the base repo's privileges.

2 It poisoned the Actions cache that the release workflow later restored.

3 Inside the real release job, it read the OIDC token from runner memory and published.

84 versions · 42 packages · 6 minutes · no token stolen · valid SLSA provenance

Harden the pipeline

- permissions: {} by default
- No PR code in pull_request_target
- No cache shared with release
- Pin actions to commit SHAs
- npm ci --ignore-scripts to publish
- Protected environments to deploy

Blast radius = lifetime × scope

Credential	Lives	Reaches	Use it for
Classic PAT	Can be forever	Every repo you can reach	Nothing. Retire them
gh CLI OAuth token	Until revoked	Your whole account	Interactive use only
Fine-grained PAT	Expiry required	Chosen repos and permissions	Personal scripts
GitHub App token	1 hour	The app's installed repos	Automation
OIDC credentials	The run	What the trust policy allows	Deploy and publish

Your PAT shouldn't do the team's job

The smell

A person's PAT, saved as a repo secret, running the release bot. It carries all their access, breaks when they leave, and rarely expires.

Fix: a GitHub App, with 1-hour tokens and its own identity, or GITHUB_TOKEN with explicit permissions.

npm already moved

- Dec, 2025: all classic tokens revoked
- npm login: a 2-hour session
- Write tokens: 90 days max, 7 by default
- Trusted publishing via OIDC

The worm's npm validator hunted for tokens with bypass_2fa: true and org write scope.

If your agent can read a secret, assume it can leak it.

Three ways out through an agent

01 · Accidental

Context

It reads .env to debug a failing test. Now the value is in the transcript, the logs, maybe the next commit.

GitGuardian: 3.2% of Claude Code-assisted commits leaked a secret.

02 · Adversarial, live

Hijack

Someone else drives it: a postinstall calls it with bypass flags, or a prompt injection in a README steers it.

s1ngularity used --dangerously-skip-permissions, --yolo and --trust-all-tools.

03 · Adversarial, planted

Persistence

Repo config runs code when you open the project: session hooks, folder-open tasks, MCP servers.

Mini Shai-Hulud committed hooks and tasks, signed as claude.

Boundaries, not instructions

Layer	Stops	Does not stop
Instructions (CLAUDE.md, AGENTS.md)	A well-meaning model	Anything an attacker writes
Permission deny rules	Claude's own Read and Edit tools	cat .env run through Bash
OS sandbox: filesystem	Every subprocess reading ~/ or .env	The built-in file tools, so use both layers
OS sandbox: network allowlist	Exfiltration to hosts you didn't allow	Abuse of a broad domain like github.com
Credential deny or mask	Reading token files and env vars	What the token can do on allowed hosts
Dev container or VM	Your credentials existing there at all	Egress, unless you limit it too

https://code.claude.com/docs/en/permissions

Demo 2

```
"sandbox": {
  "enabled": true,
  "failIfUnavailable": true,
  "allowUnsandboxedCommands": false,
  "filesystem": {
    "denyRead": ["~/", "./.env"],
    "allowRead": ["."]
  },
  "network": {
    "allowedDomains": ["registry.npmjs.org"]
  },
  "credentials": { "envVars": [
    { "name": "GH_TOKEN", "mode": "deny" } ] }
}
```

1 · "Show me .env"

Denied by a permission rule

2 · cat .env

Blocked by the OS sandbox

3 · npm run demo:install-unsafe

Same package now finds nothing

4 · curl example.com

Not on the allowlist

5 · Masked GH_TOKEN

Usable, not readable (recorded)

Agent config is code. Review it.

```
# Agent and CI config needs security review
/.claude/ @rabbit/security
/.vscode/ @rabbit/security
/.mcp.json @rabbit/security
/CLAUDE.md @rabbit/security
/AGENTS.md @rabbit/security
/.github/workflows/ @rabbit/security
```

In review, look for

- Any hooks key in .claude/settings.json
- runOn: folderOpen in .vscode/tasks.json
- New MCP servers in .mcp.json
- Wider allowRead or allowedDomains
- Bot-looking commits: the worm signed as claude@users.noreply.github.com

MCP configs: reference env vars, never paste keys. 24,008 leaked from public repos.

A check at every hop

Agent writes

Scan changes before commit, e.g. GitHub MCP secret scanning

Commit

Pre-commit: gitleaks, ggshield, TruffleHog

Push

Push protection on every repo

CI

Scan PRs and full history; watch runner egress

Install

ignore-scripts and a release-age cooldown

Cooldowns work

Most malicious versions are pulled within hours. TanStack's were caught in about 20 minutes.

Honeytokens tell you

A canary key where only a leak would find it. The one signal a deny rule can't give you.

Demo 3: Hook catches the agent

```
$ npm run demo:fake-leak
  # the "agent" hardcodes a GitHub token (fake, random)
$ git add demo/agent-written-config.js
$ git commit -m "fix api client"
  RuleID: github-pat
  Commit blocked by gitleaks
```

Agent commits run the same hooks yours do. That is the point.

When one slips: revoke first

1

Revoke

At the provider, before anything else. Public means already scraped.

2

Check for use

Audit logs from the moment of exposure. Assume use until proven otherwise.

3

Rotate the reach

Anything that credential could read is exposed too.

4

Hunt persistence

Hooks, folder-open tasks, new workflows, repos and versions you didn't make.

5

Clean history, second

filter-repo and cache purges reduce future exposure. They un-leak nothing.

6

Write it up

Which hop failed, and which guardrail would have caught it.

TAKE IT HOME

Fifteen checks for your repo

Local

- References, not values
- ignore-scripts and a release-age cooldown
- Boring dev credentials

CI/CD

- OIDC, no stored keys
- sub pinned to an environment
- No PR code in privileged jobs

Tokens

- No classic PATs
- A GitHub App for Automation
- The agent gets its own identity

Agents

- Sandbox on, home denied
- Tokens denied or masked
- CODEOWNERS on agent config

Safety nets

- Pre-commit and push protection
- A honeytoken
- A revoke-first runbook

If your agent can read a secret, assume it can leak it.

So give it less to read.

THANK YOU TO OUR

SPONSORS

PLATINUM







GOLD







SILVER



Questions?

Jitendrudu · jitendrudu@hotmail.com · X: @LatLon_

Research credit: StepSecurity, TanStack, GitGuardian, Cloud Security Alliance

